

Mini Workshop

Working with a Linux shell

Andrea Telatin, Quadram Institute Biosciences





Join Discord first

<https://discord.gg/FUxH9Qz6f>

► Websites

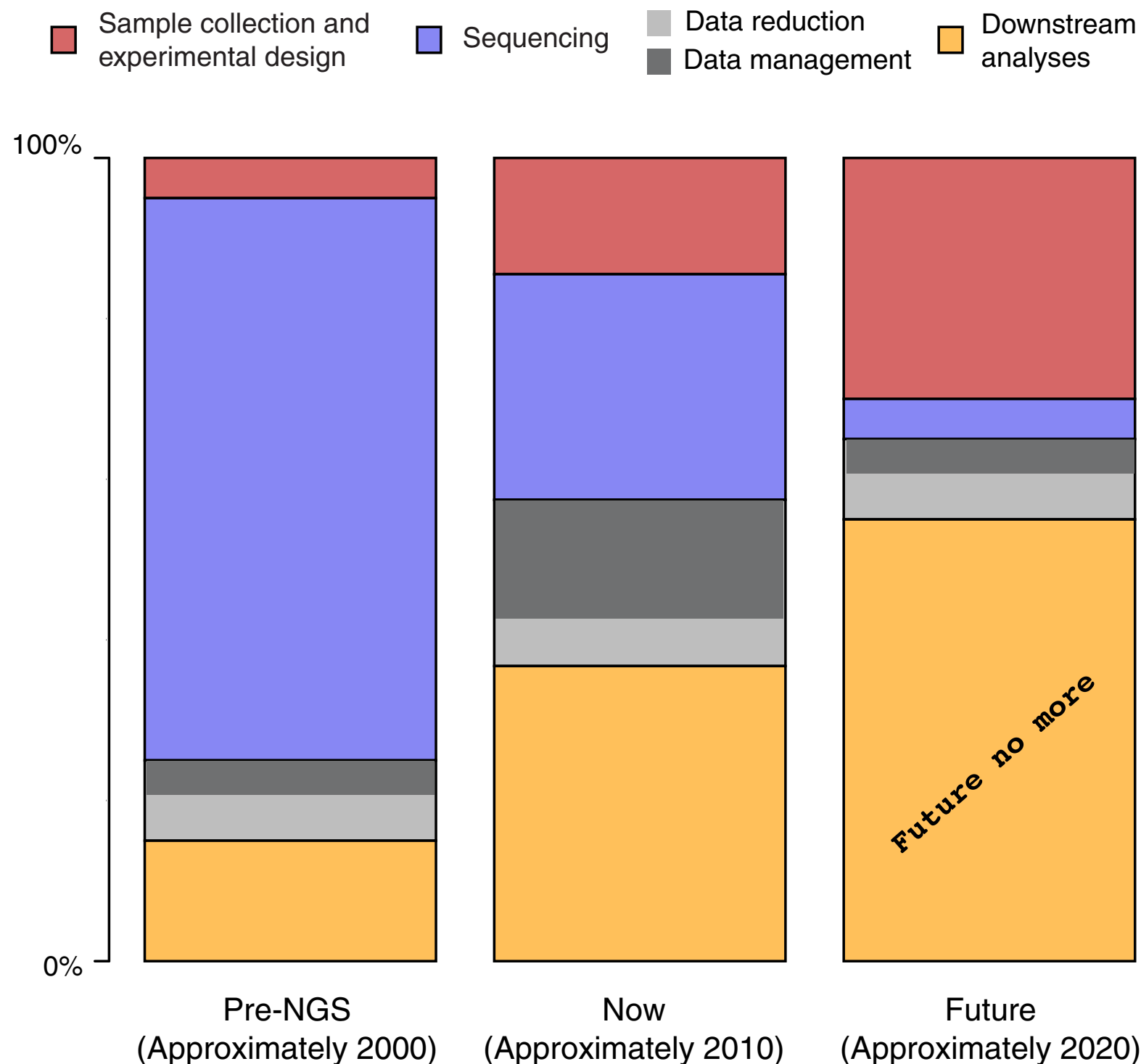
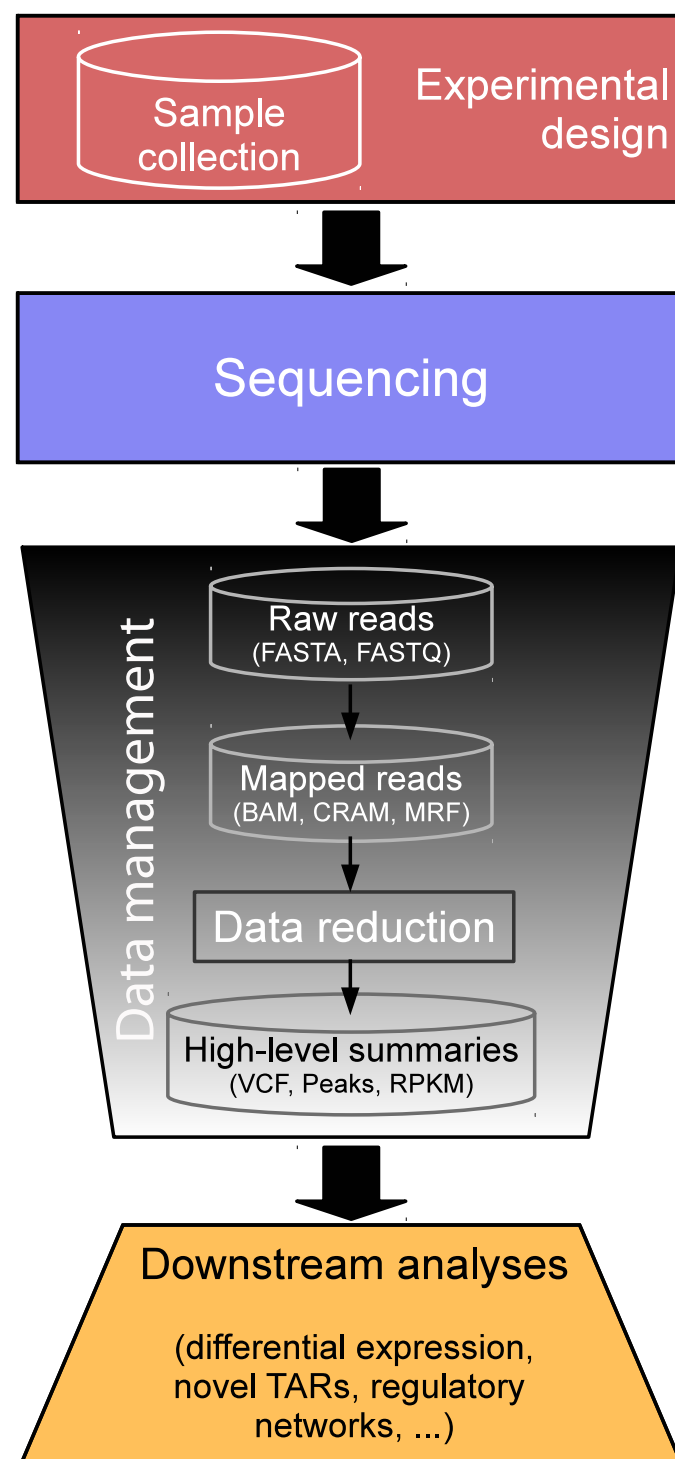
- <https://telatin.github.io/microbiome-bioinformatics/Bash-course/>
- https://github.com/telatin/learn_bash/

► Program

- Working with **Linux** and its **filesystem**
- **Text files** and bioinformatics file formats
- Bioinformatics **programs** and how to find them

► Goals

- Demystifying the use of the command line
- Understanding why some tasks are better done from the command line
- Understanding how files and folders are accessed from the command line
- Understanding what simple text files are, and their use in bioinformatics
- Seeding curiosity to learn more :)





A yellow parallelogram graphic, tilted to the right, positioned behind the text.

what is a shell?

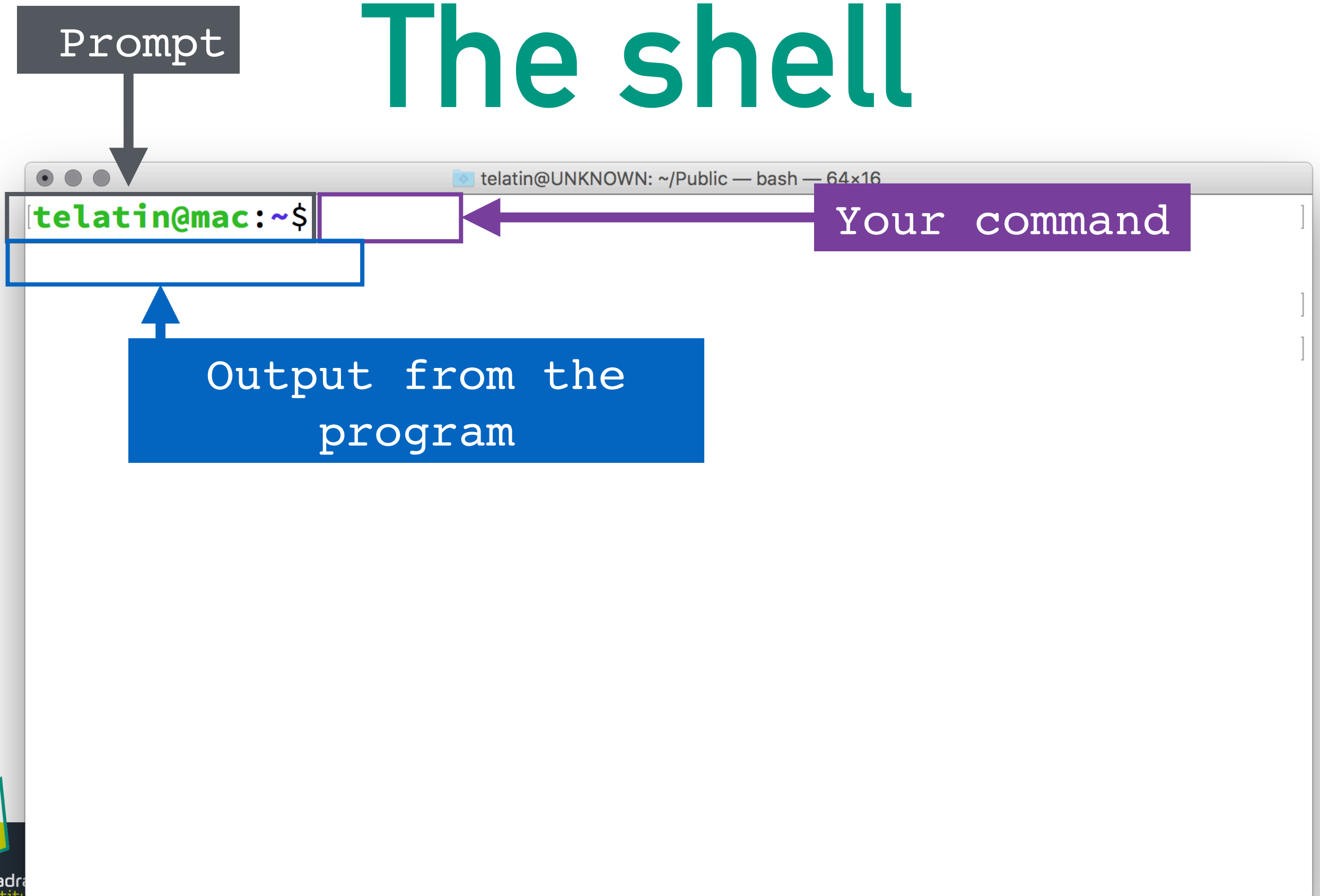
The UNIX Terminal

- it's a **powerful text interface** to interact with PCs and servers
- carries a long history of **evolution** and improvements from the '70
- it's **everywhere**: most websites are hosted in a Linux server. Any Mac runs on UNIX.
It matters to us: bioinformatics loves Linux.

The shell

- It's a program that allows us to interact with the system: list files, run other programs...
- DOS was a shell by Microsoft. We are interested in UNIX compatible shells, there are many the most commonly found in servers being **Bash**
- Bash can replace whatever you are used to do under Windows with a mouse, and more.

The shell



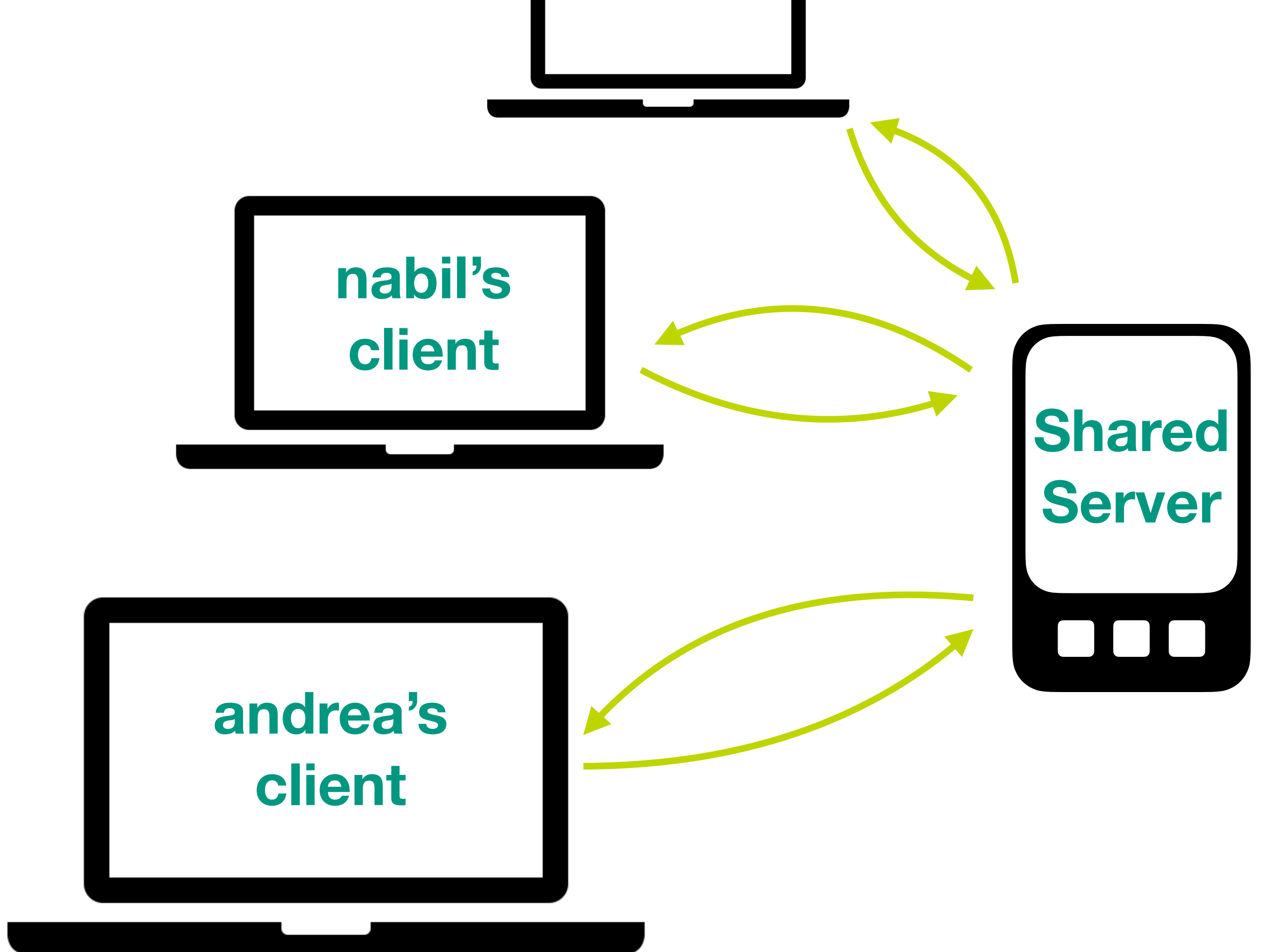
The shell

```
telatin@UNKNOWN: ~/Public — bash — 64x16
[telatin@mac:~$ pwd
/Users/telatin
[telatin@mac:~$ cd Public/
[telatin@mac:~/Public$ ls -l
total 8
drwxr-xr-x  14 telatin  staff  476 Nov 17  2015 geeknote
-rw-r--r--   1 telatin  staff   18 Apr  7 08:04 readme.txt
telatin@mac:~/Public$
```

- ▶ How many commands are present?
- ▶ Did some program return no output?



remote servers



Benefits: servers are...

- **shared resources:** multiple users can access the same data (reducing the need and risks of duplications)
- **powerful resources:** servers carry more computational power for the same cost of a laptop
- **reliable resources:** redundant power supply, redundant disks. Higher availability and data is more secure
- **managed resources:** we don't need to remember to backup our data, someone else will.

Connecting to a remote server

→ From Linux or macOS

- Open your terminal
- Type `ssh remoteuser@remoteaddress`

→ From Windows

- You need a terminal emulator, for example a program called **PuTTY**

Connecting to a remote server

→ From

```
telatin@mac:~$ pwd
```

```
/Users/telatin
```

```
telatin@mac:~$ ssh ubuntu@robin.seq.space
```

- ubuntu@robin.seq.space's password: !

```
=====
```

- Welcome to your Galaxy on the Cloud instance.

```
=====
```

→ From

```
* Documentation: https://wiki.galaxyproject.org/CloudMan
```

- *** System restart required ***

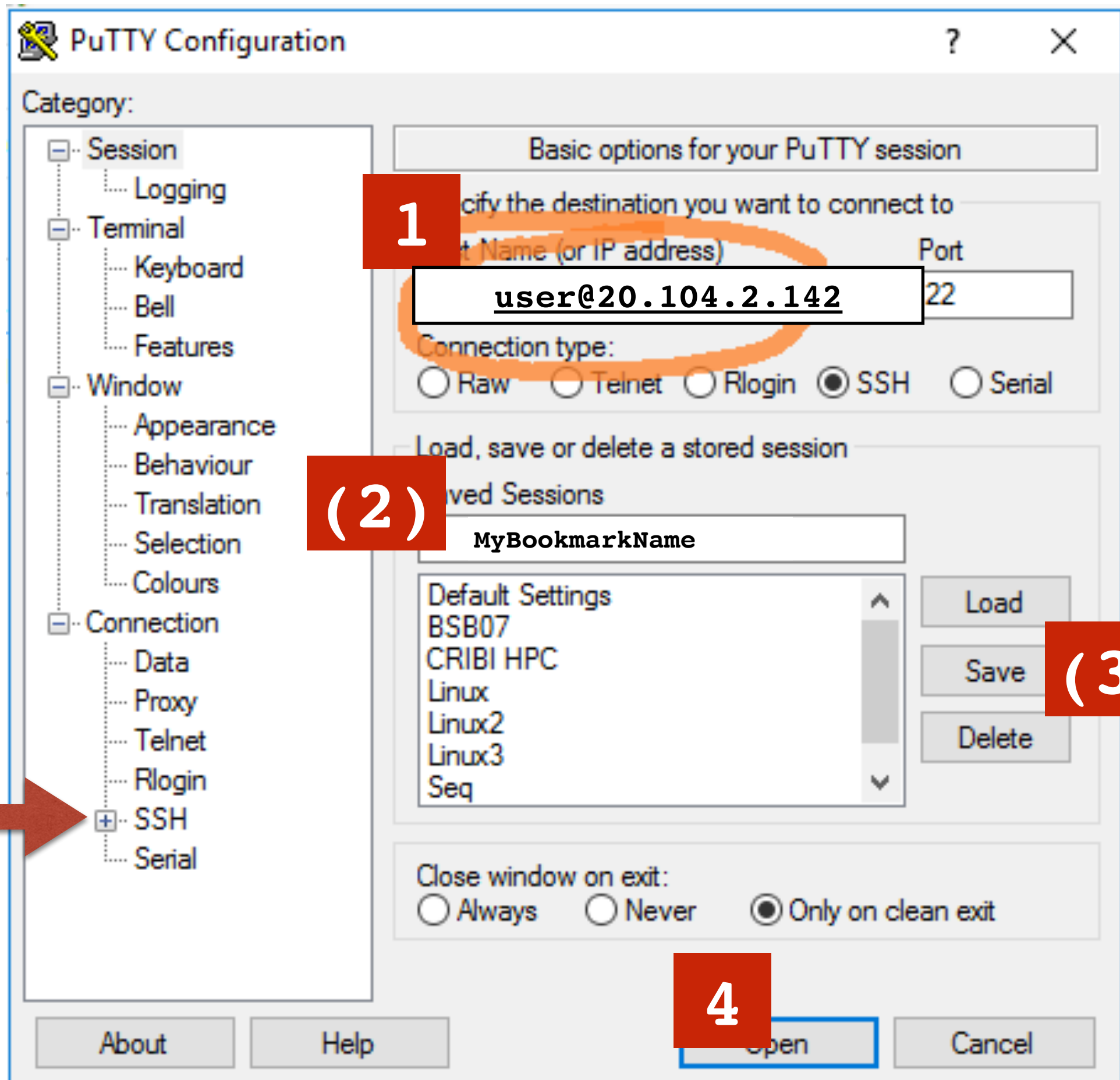
```
ubuntu@robin:~$ pwd
```

```
/home/ubuntu
```

```
ubuntu@robin:~$
```



putty.exe

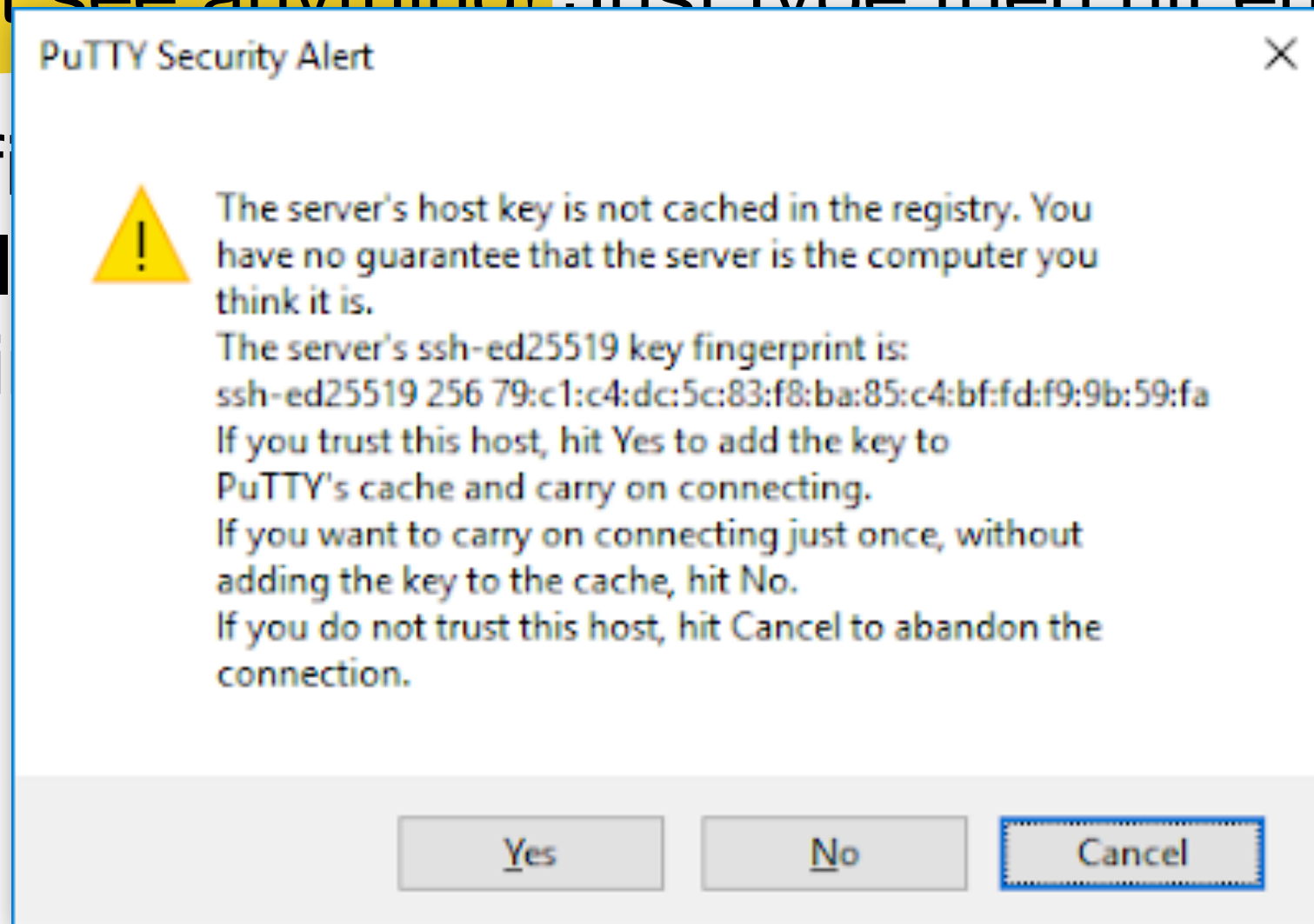


putty.exe

Connecting to a remote server

- When typing password inside a terminal you won't see anything. Just type then hit enter!

- The first time you'll connect to a new server you'll see a warning dialog box.



ver
the

What files will I see?

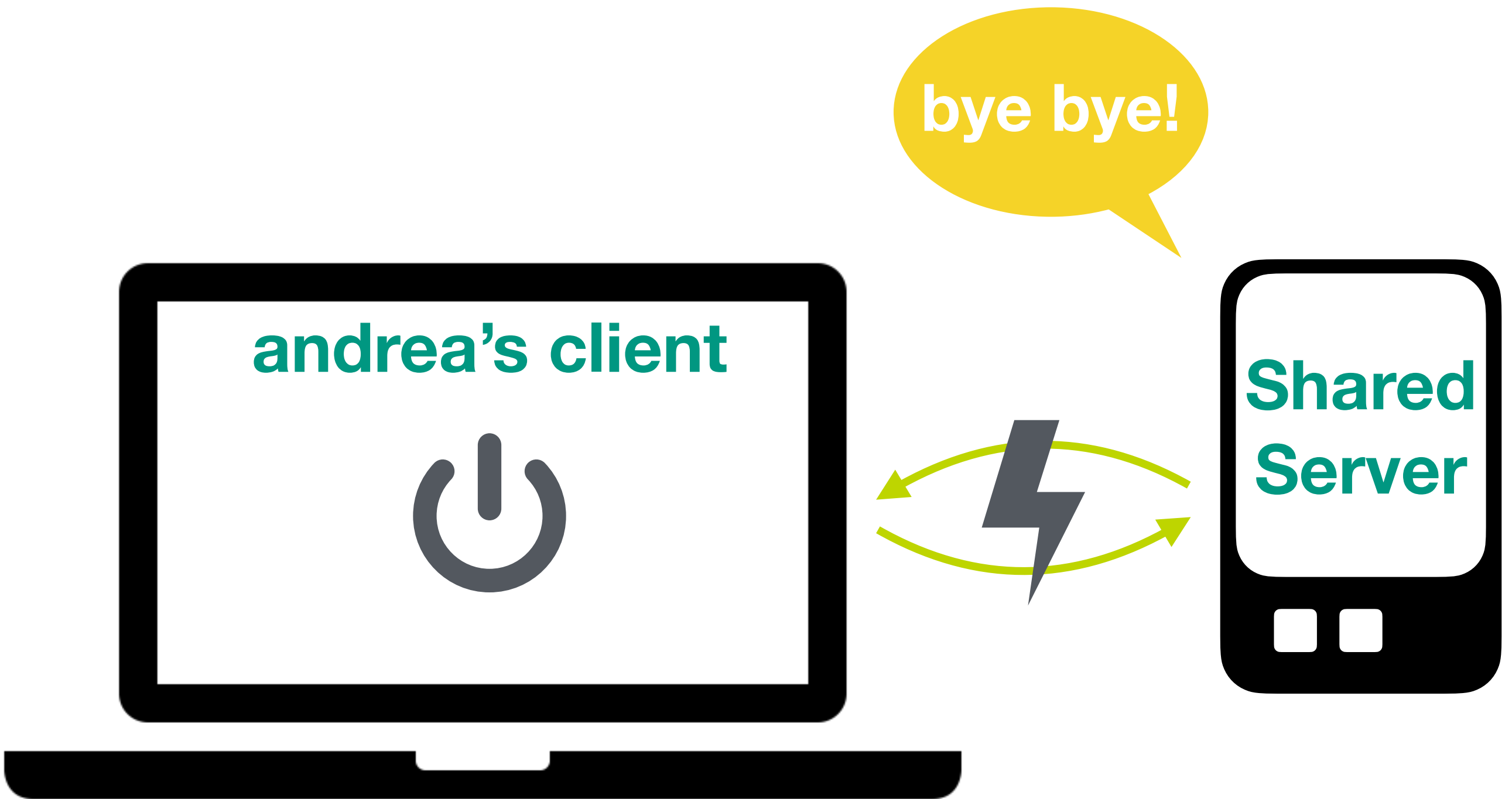
- Once logged into the remote server you'll only be able to see the files present there
- It's possible to copy files from your local machine (client) to the remote machine (server) using FileZilla or WinSCP
- It's a good idea to **always** use some server for our work: clients are never as safe and reliable as a server.



“screen” safety
net



SSH Session: a constant flow of text going to
and coming from the remote server



Even a small interruption in the connectivity will make the server assume we are no longer interested!



We can use a program called **screen** (or another called **tmux**) to keep the session alive even if we close our computer!

The “screen cycle”

- Check if there are screens already
`screen -list`
- Resume the existing screen (if present)
`screen -dr`
- Create a new screen otherwise
`screen -S SessionName`

Ensure you are not creating unneeded sessions

● ● ●  andrea — -bash — 80×24

Last login: Mon Apr 9 12:10:59 on ttys001

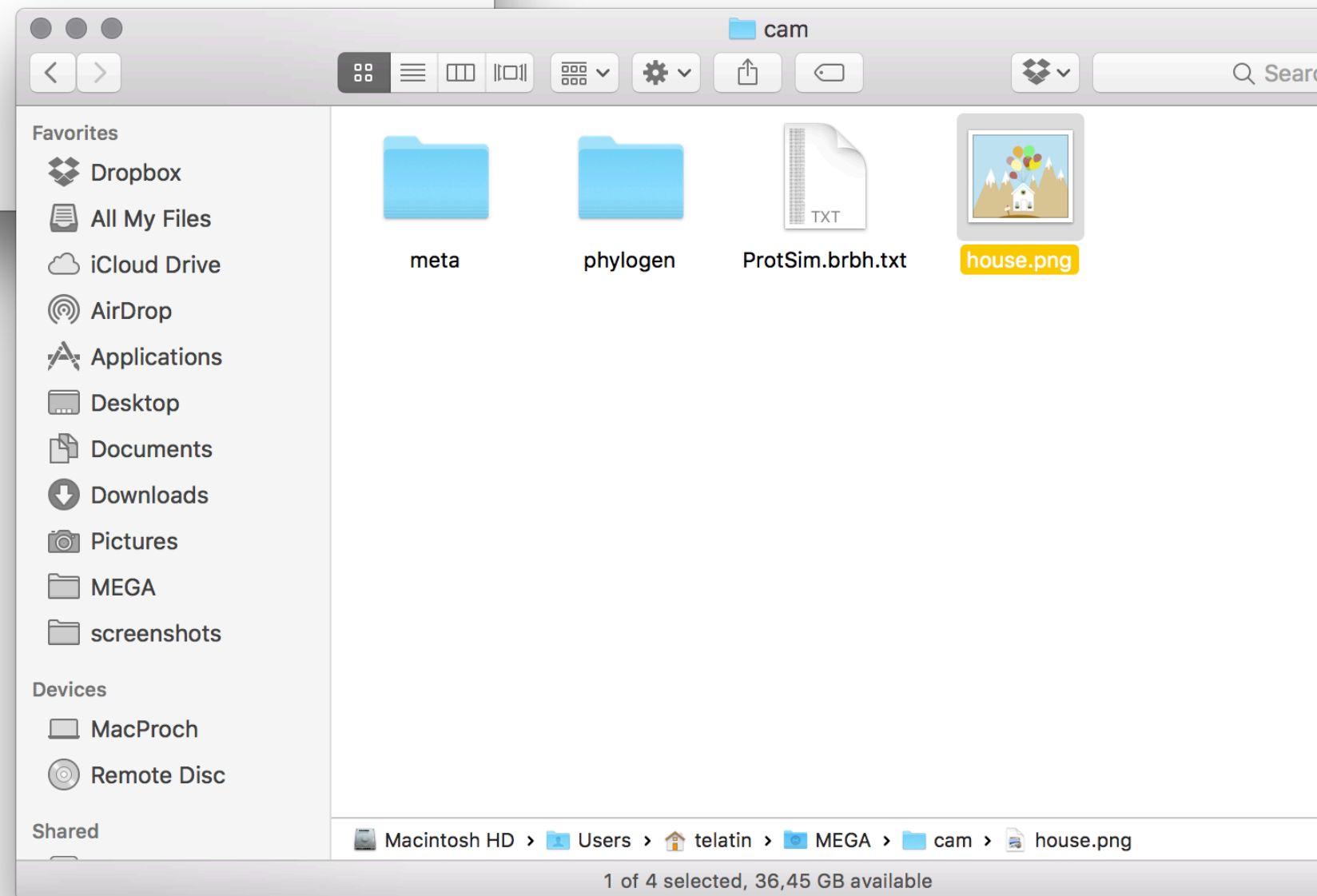
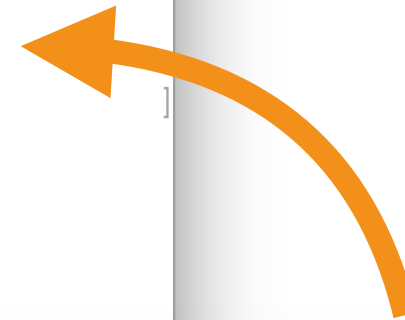
andrea@mymac ~ \$ p



File Paths

a.k.a. the most difficult topic of this course!

```
cam — telatin@MacProch — ~/MEGA/cam — zsh — 66x18
[→ cam pwd
/Users/telatin/MEGA/cam
[→ cam ls -l
total 480
-rw----- 1 telatin staff 51497 Jan 26 2016 ProtSim.brbh.txt
-rw-r--r--@ 1 telatin staff 188745 Aug 6 16:13 house.png
drwxr-xr-x 3 telatin staff 102 Jan 29 2016 meta
drwxr-xr-x 5 telatin staff 170 Jan 29 2016 phylogen
[→ cam ls meta
ecoli_iJ01366.tsv
[→ cam
```

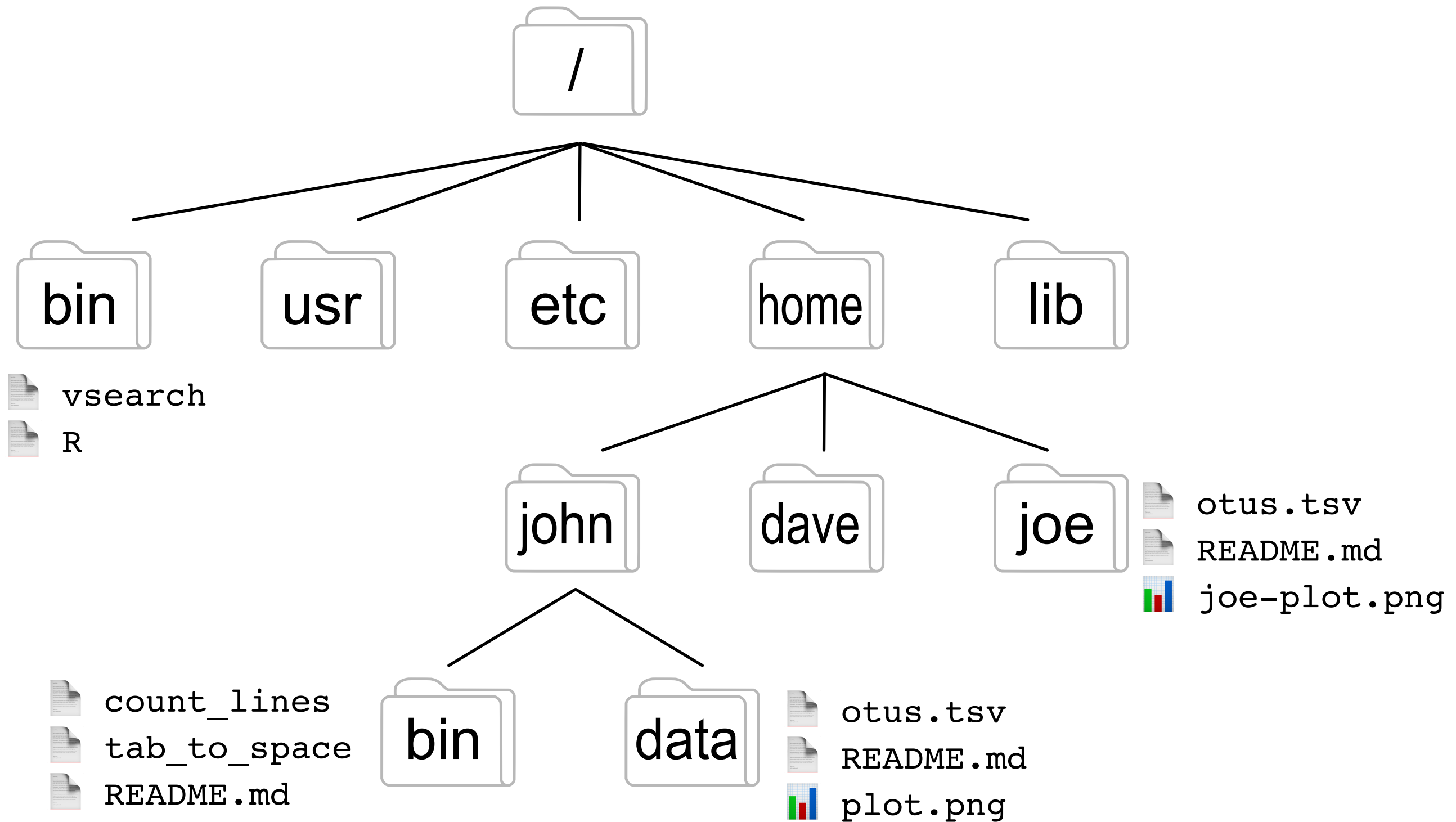


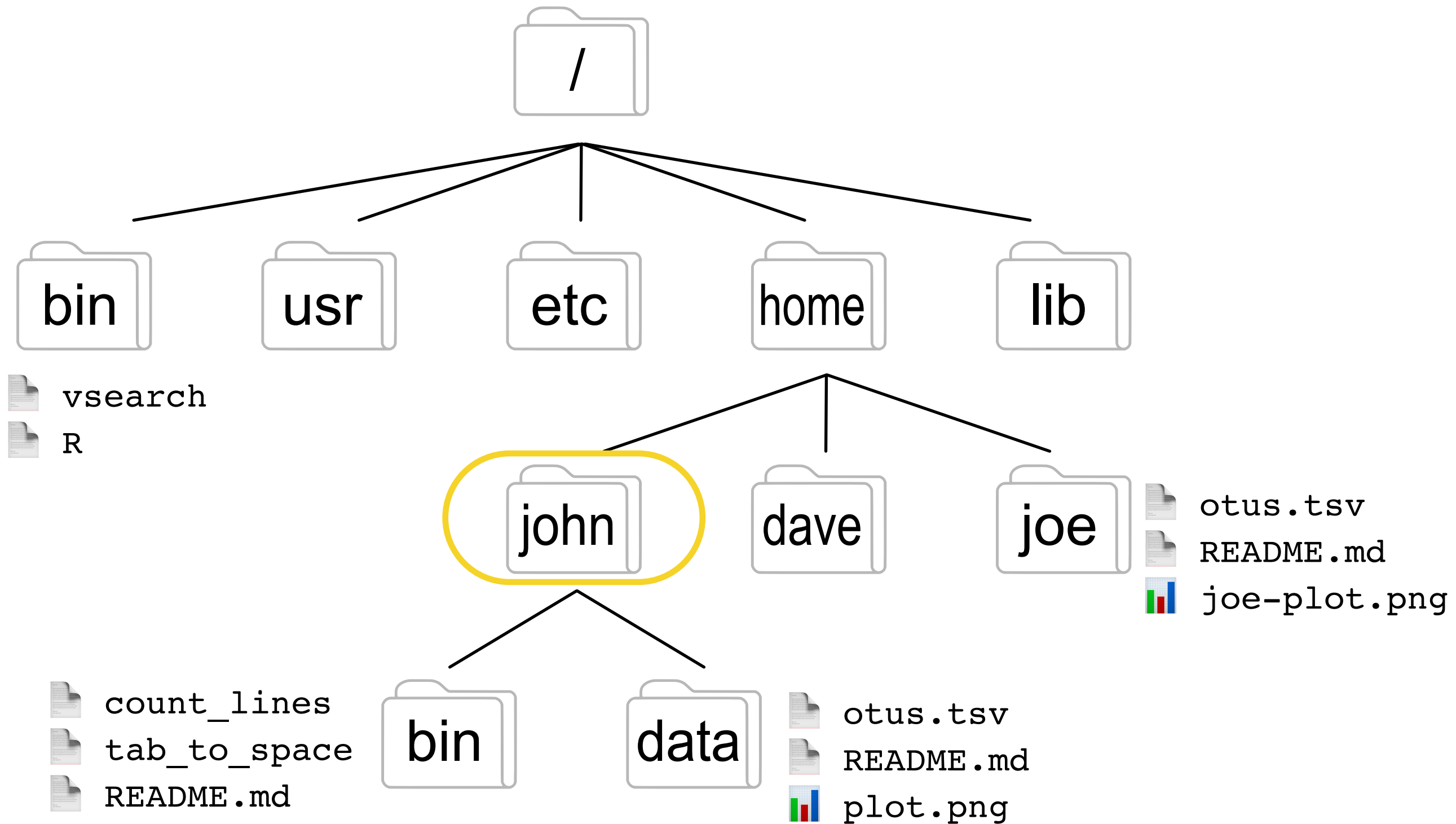
Concepts

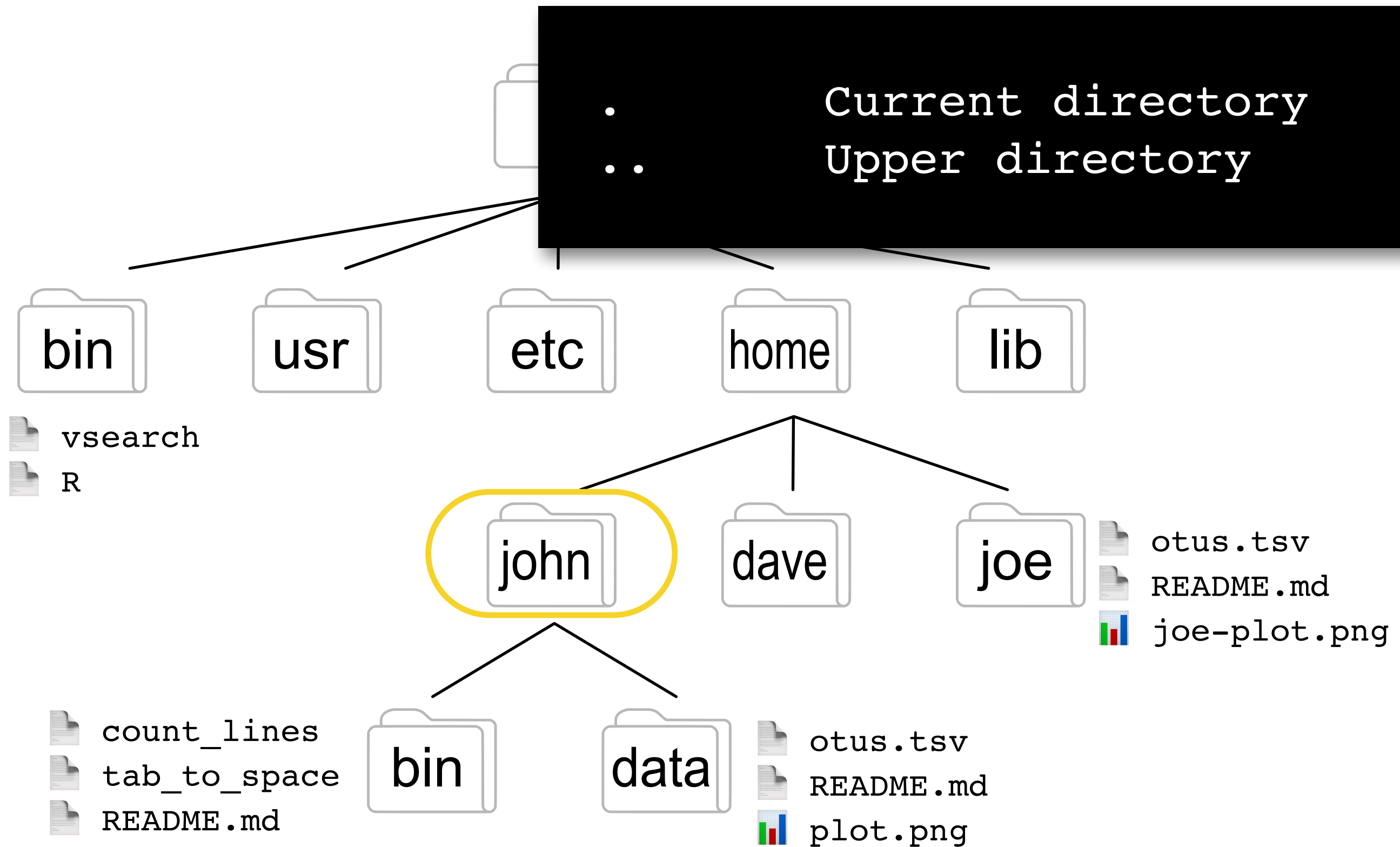
- **Current directory**, or **working directory**, is the place you are currently operating from. You always are operating from a specific directory.
- Each file has a unique position in the filesystem tree (like a leaf in a taxonomy tree). You can describe it in two ways: **absolute** or **relative**.

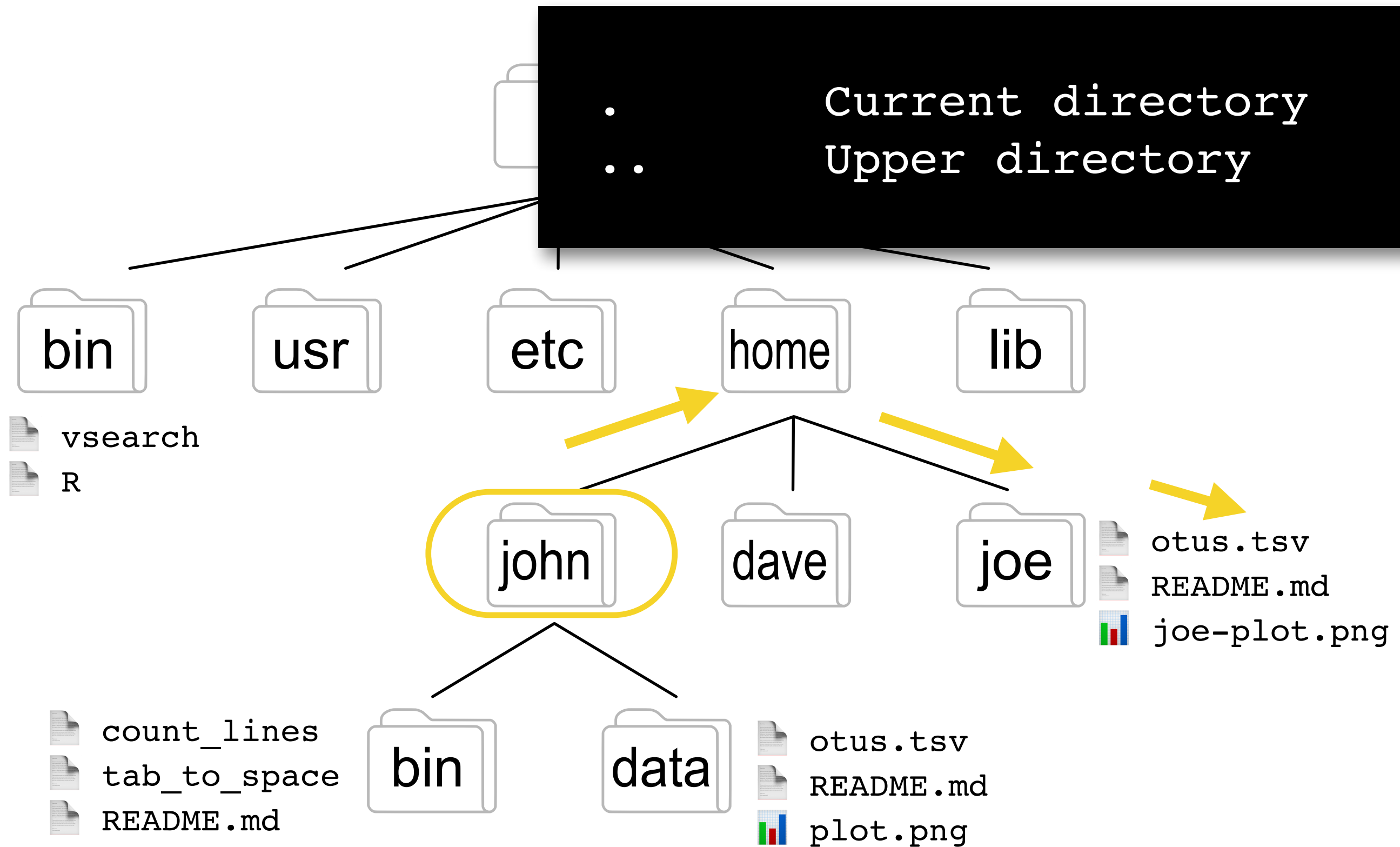
Concepts

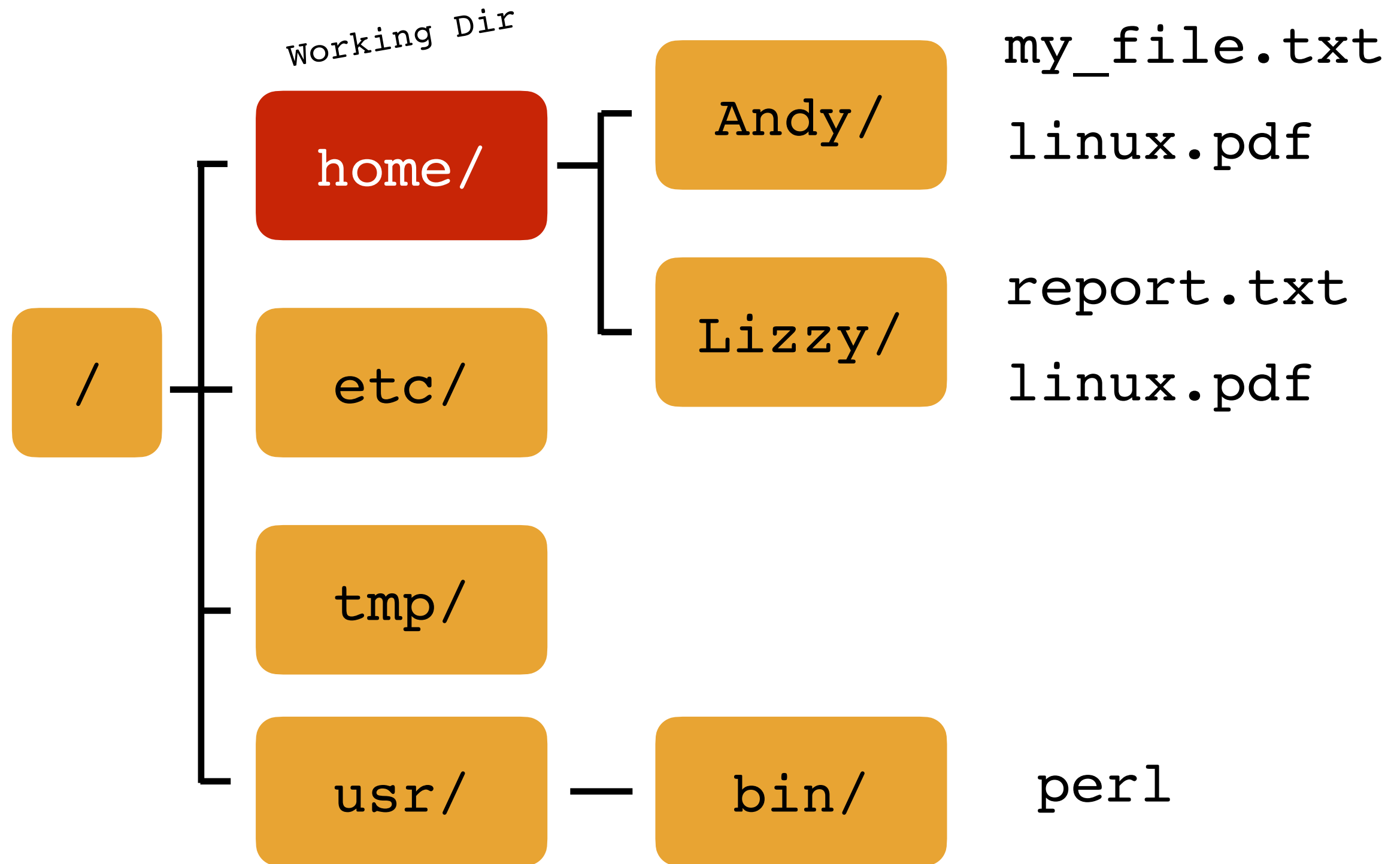
- **Absolute paths**, like absolute coordinates, are the same for any user of the system, no matter the current working directory. They always start with /
- **Relative paths** are generated assuming your working directory as the starting point. They never start with /.





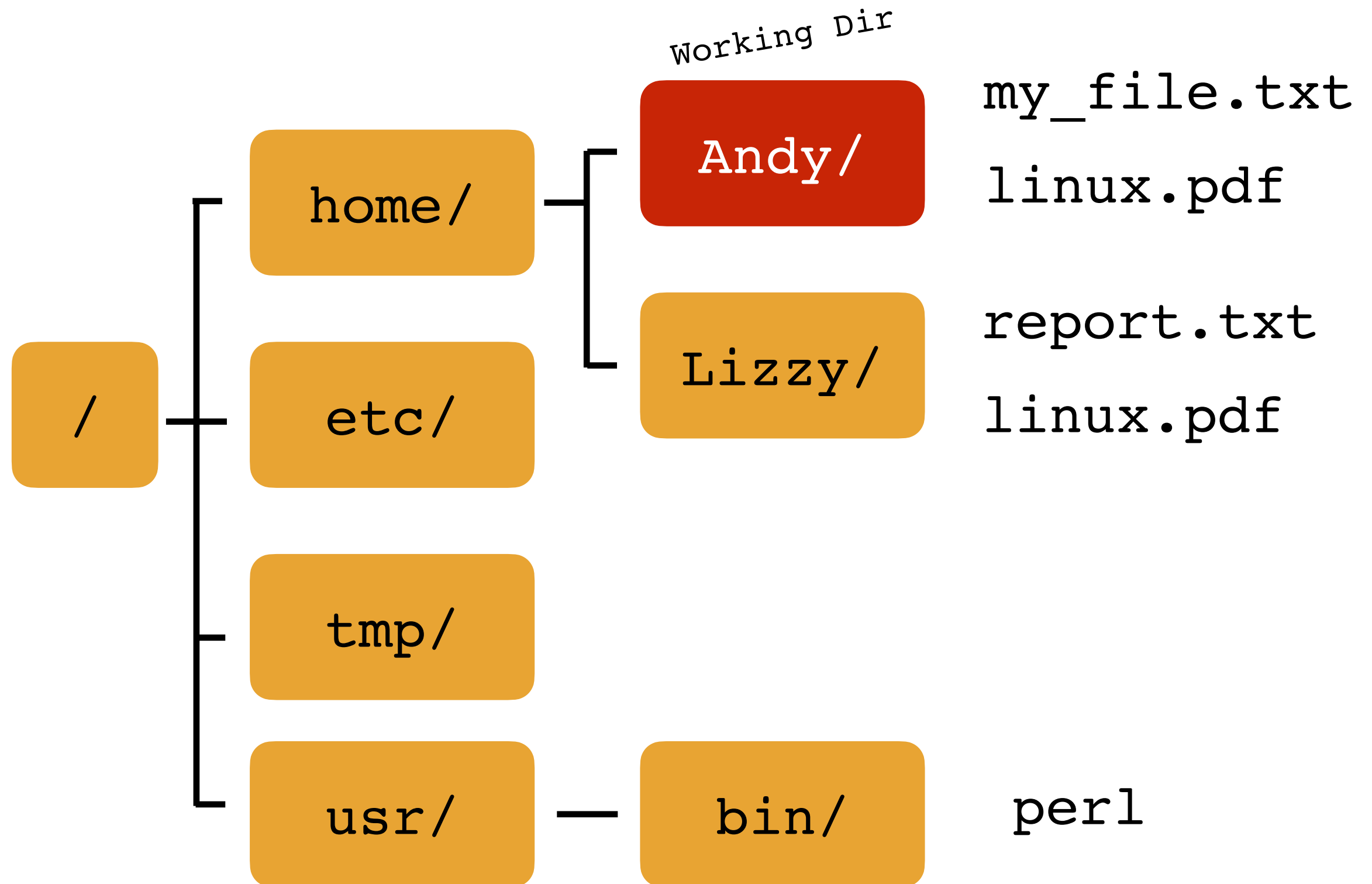


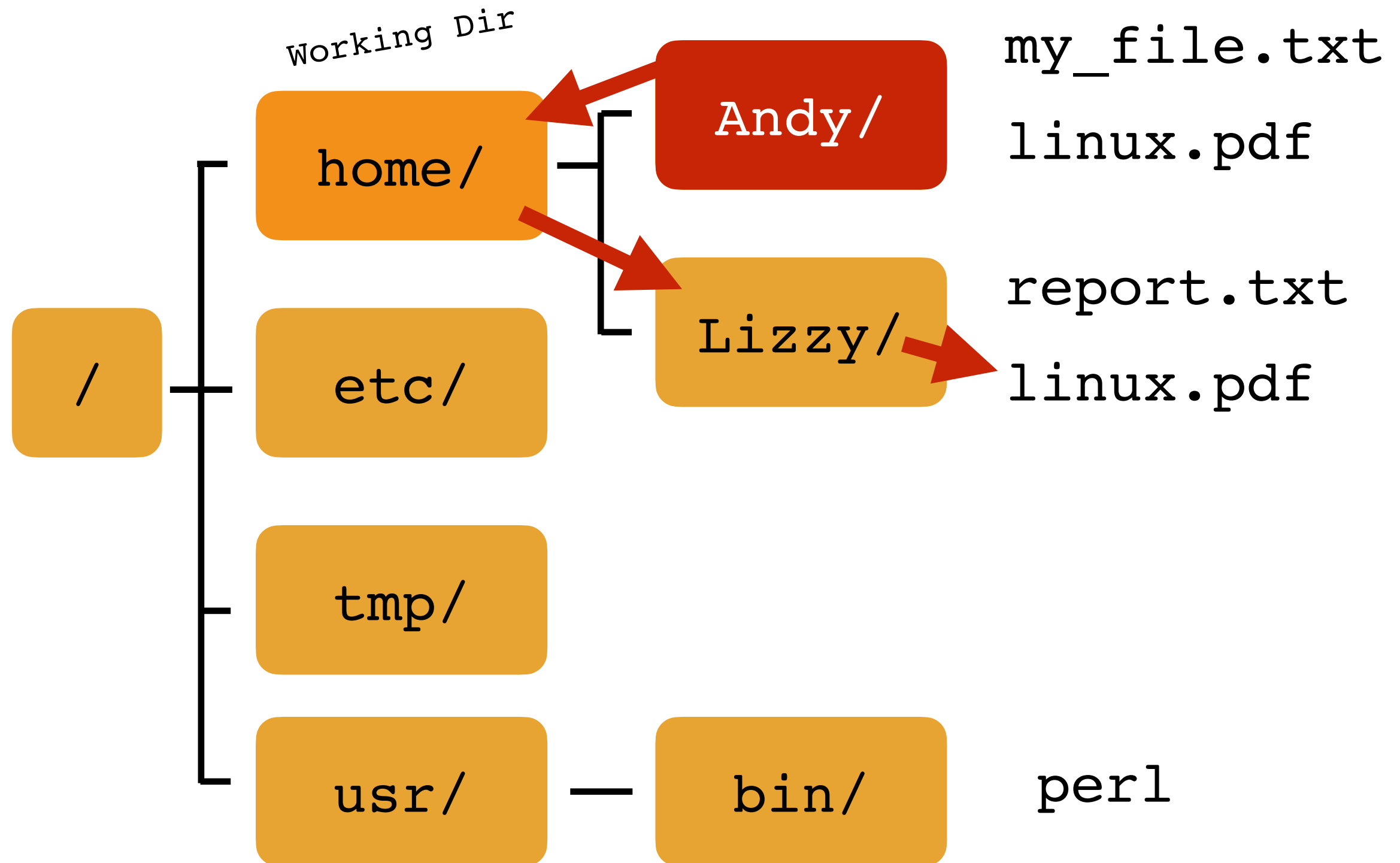


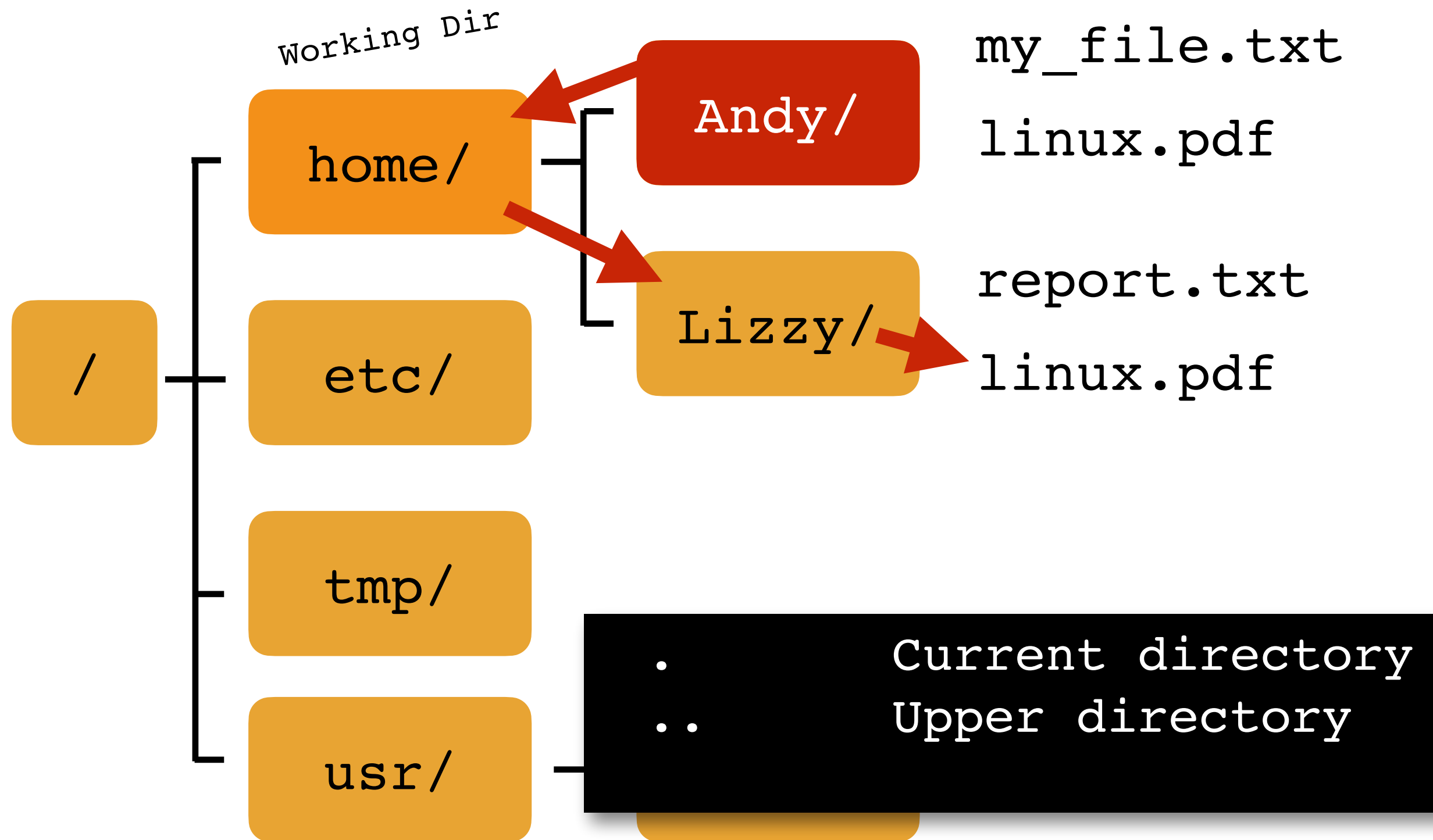


```
cd ./Andy
```

```
cd /home/Andy
```







```
ls -l ../Lizzy/linux.pdf
```

Commands history

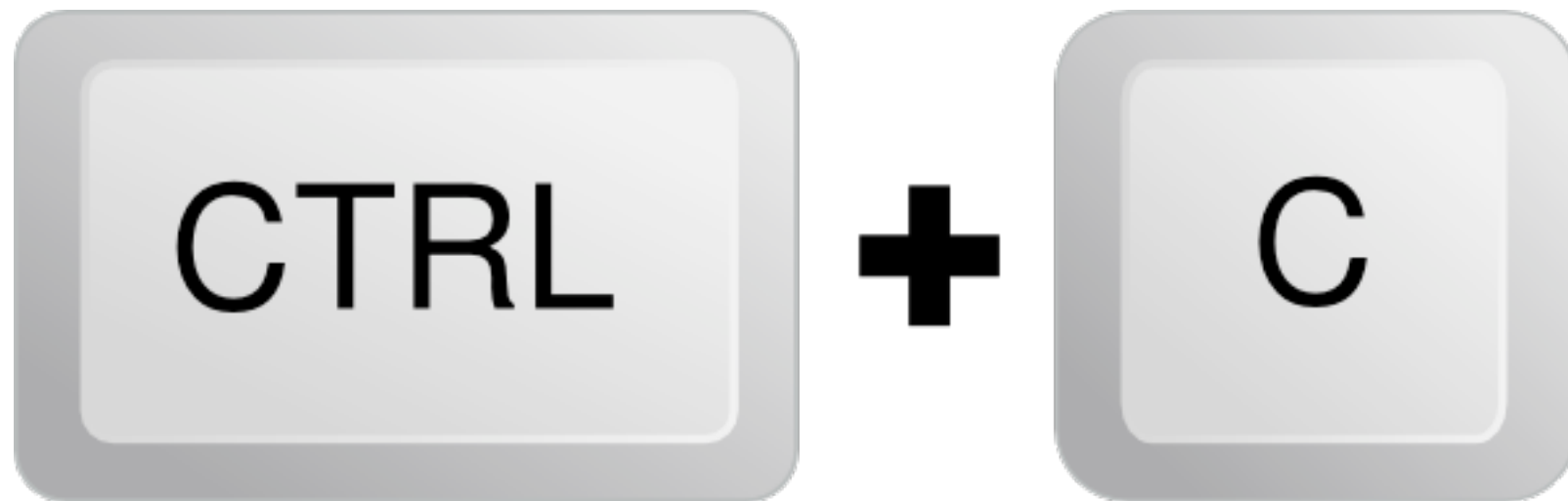
- Using the **up arrow** you will recall the last command typed, hit it again and you get the previous and so on.
- Down arrow is working as well
- The command **history** will print the list of commands issued so far



Tab completion

- When you are typing a file path (relative or absolute) you are likely to introduce errors.
- Most shells have a key (Tab) to help you type a correct path.
- It's thus **mandatory** to use the  autocompletion

Panic management



- Breaks current work (**exit**)
- After killing the current process, you'll get the prompt back
- Also remember that **Ctrl + D** breaks a stream of data



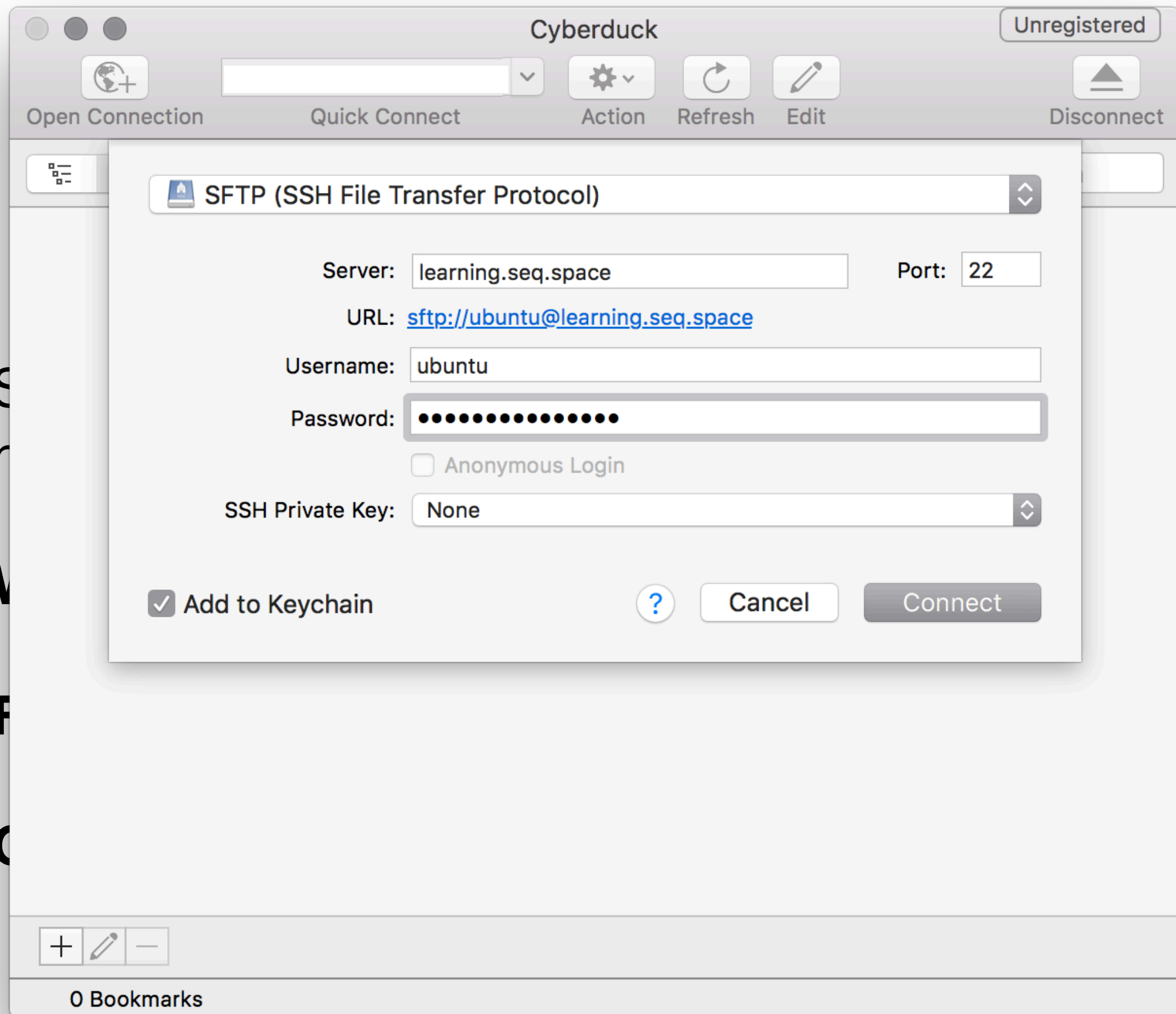


File transfers

Graphical way

- Same authentications as `ssh` (you'll need server name, username and password)
- **WinSCP** for Windows
- **FileZilla** for macOS and Linux
- **Cyber Duck**, macOS and Windows





wiki - My Server - WinSCP

Local Mark Files Commands Session Options Remote Help

Synchronize Queue Transfer Settings Default

My Server Work New Session

D: Data wiki

Upload Edit Properties Download Edit Properties

D:\Documents\wiki /home/martinp/httpdocs/wiki/wiki

Name	Ext	Size	Changed	Attr	Name	Ext	Size	Changed	Rights
interfaces.txt		1 611	14.10.2011 22:02:00	a	..			31.10.2011 23:03:09	rw-r-xr-x
introduction.txt		1 662	25.10.2011 23:13:26	a	wiki			6.5.2012 21:47:48	rw-r-xr-x
languages.txt		2 362	9.9.2011 15:03:29	a	.htaccess		33	10.11.2004 21:46:46	rw-r--r--
operation_mask.txt		2 035	25.4.2005 11:35:48	a	commandline.txt		9 132	1.4.2012 11:53:47	rw-r--r--
protocols.txt		5 341	30.7.2011 22:14:42	a	contributions.txt		1 798	28.8.2011 7:14:44	rw-r--r--
remote_command...		2 543	11.9.2011 21:01:44	a	directory_cache.txt		1 892	30.7.2011 22:24:53	rw-r--r--
requirements.txt		5 608	8.10.2011 21:24:24	a	dragext.txt		4 521	4.12.2011 22:19:32	rw-r--r--
resume.txt		3 208	30.7.2011 22:45:09	a	faq.txt		6 375	9.3.2012 9:09:44	rw-r--r--
screenshots.txt		879	9.4.2008 11:29:58	a	faq_commandlin...		102	17.12.2004 11:45:36	rw-r--r--
scripting.txt		8 678	1.11.2011 15:19:57	a	faq_dir_default.txt		1 120	24.5.2011 11:17:20	rw-r--r--
security.txt		1 288	16.8.2011 22:00:51	a	faq_download_te...		751	21.11.2005 8:39:25	rw-r--r--
shell_session.txt		1 216	30.7.2011 23:03:27	a	faq_drag_move.txt		1 554	17.9.2010 9:34:23	rw-r--r--

22 706 B of 87 797 B in 8 of 33 17 407 B of 1 069 KiB in 4 of 315

Queue (3)

Operation	Source	Destination	Transferred	Time/Speed	Progress
?	/home/martinp/httpdocs/wiki/...	D:\Documents\wiki*.*	11 307 B	0:00:00	Query
	faq.txt		0 B	18 985 B/s	0%
	D:\Documents\movies\Movi...	/home/martinp/httpdocs/*.*	212 MiB	0:00:09	80%
	D:\Documents\movies\Movi...		212 MiB	5 566 KiB/s	80%
	/home/martinp/httpdocs/for...	D:\Documents\backup*.*			Waiting...

SFTP-3 0:10:25



Anatomy of a command



Structure of a command

- **command** parameters ...



Structure of a command

- **command** parameters ...

- `ls` `-l` `/workshop`



- `grep` `-c` `">"` `file-name.txt`



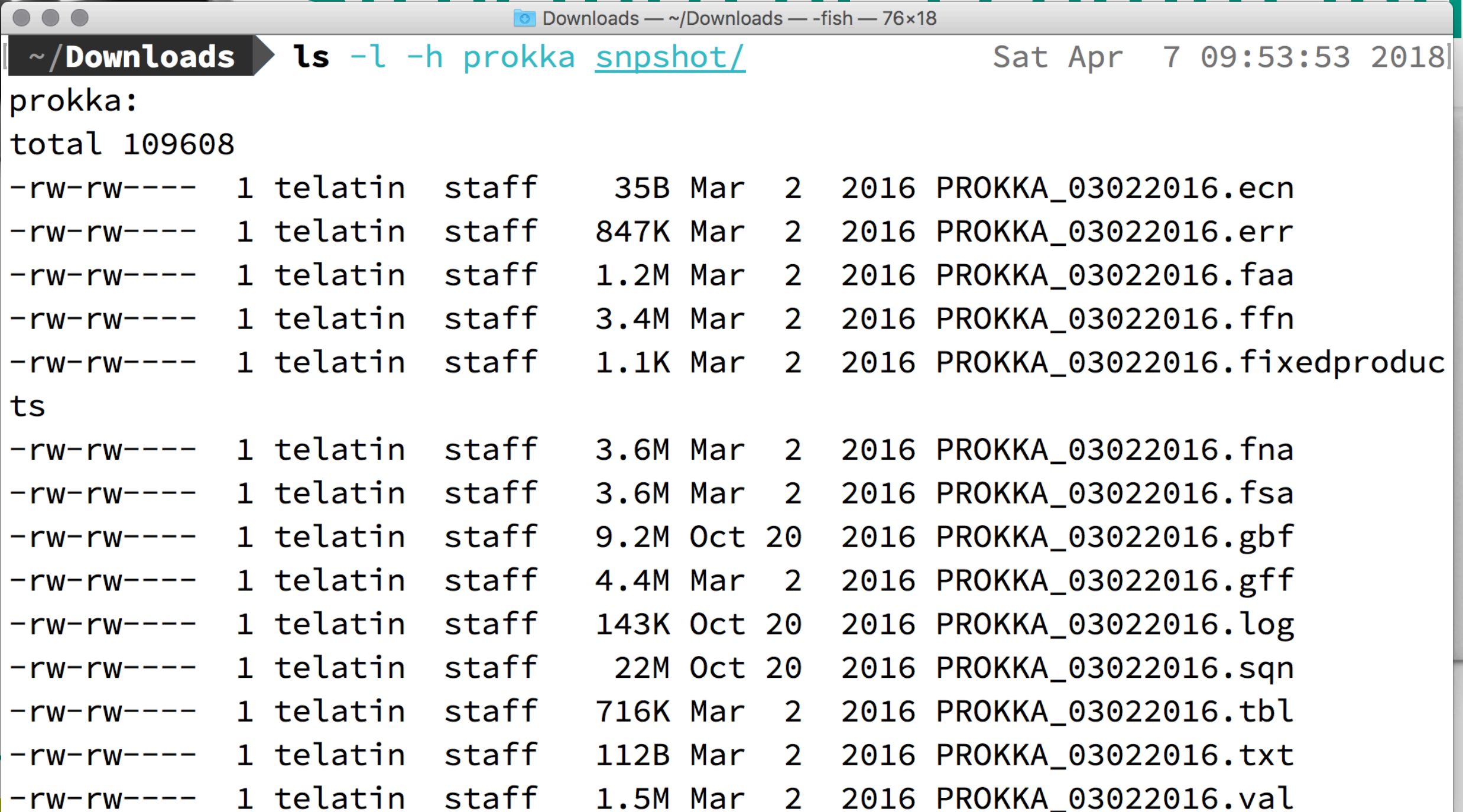


Structure of a command

• `spades.py` `-s` `file-name.txt` `-t` `8` `--careful`



Structure of a command



A terminal window titled "Downloads — ~/Downloads — -fish — 76x18" displays the command `ls -l -h prokka snpshot/` and its output. The output shows a directory listing for the `prokka` directory, including file permissions, owner, group, size, date, and filename. The files listed are `PROKKA_03022016.ecn`, `PROKKA_03022016.err`, `PROKKA_03022016.faa`, `PROKKA_03022016.ffn`, `PROKKA_03022016.fixedproducts`, `PROKKA_03022016.fna`, `PROKKA_03022016.fsa`, `PROKKA_03022016.gbff`, `PROKKA_03022016.gff`, `PROKKA_03022016.log`, `PROKKA_03022016.sqn`, `PROKKA_03022016.tbl`, `PROKKA_03022016.txt`, and `PROKKA_03022016.val`.

```
~/Downloads ▶ ls -l -h prokka snpshot/ Sat Apr 7 09:53:53 2018
prokka:
total 109608
-rw-rw---- 1 telatin staff 35B Mar 2 2016 PROKKA_03022016.ecn
-rw-rw---- 1 telatin staff 847K Mar 2 2016 PROKKA_03022016.err
-rw-rw---- 1 telatin staff 1.2M Mar 2 2016 PROKKA_03022016.faa
-rw-rw---- 1 telatin staff 3.4M Mar 2 2016 PROKKA_03022016.ffn
-rw-rw---- 1 telatin staff 1.1K Mar 2 2016 PROKKA_03022016.fixedproduc
ts
-rw-rw---- 1 telatin staff 3.6M Mar 2 2016 PROKKA_03022016.fna
-rw-rw---- 1 telatin staff 3.6M Mar 2 2016 PROKKA_03022016.fsa
-rw-rw---- 1 telatin staff 9.2M Oct 20 2016 PROKKA_03022016.gbff
-rw-rw---- 1 telatin staff 4.4M Mar 2 2016 PROKKA_03022016.gff
-rw-rw---- 1 telatin staff 143K Oct 20 2016 PROKKA_03022016.log
-rw-rw---- 1 telatin staff 22M Oct 20 2016 PROKKA_03022016.sqn
-rw-rw---- 1 telatin staff 716K Mar 2 2016 PROKKA_03022016.tbl
-rw-rw---- 1 telatin staff 112B Mar 2 2016 PROKKA_03022016.txt
-rw-rw---- 1 telatin staff 1.5M Mar 2 2016 PROKKA_03022016.val
```

Structure of a command

Downloads — ~/Downloads — fish — 76x18
~/Downloads ▶ **ls** -l -h prokka snpshot/ Sat Apr 7 09:53:53 2018

prokka:
total 109608

telatin — ~ — fish — 58x15
~ ▶ **weather.pl** **--city "New York"** --days 3

Coffee Break





Our first commands

Where am I: `pwd`

- Means “print working directory”, no parameters to be supplied.
- It returns the **absolute path** of our current position
- It's important to check if we are where we think we are (see also `ls`)

Change directory: `cd`

- **`cd`** (Change directory) allows to set a new current directory
- Syntax: `cd new_path`
- Path can be absolute:
`/home/telatin/Desktop/directory/`
- or relative:
`../../destination`

Change directory: `cd`

- Without parameters goes to the home directory
E.g.: `cd`
- There is a shortcut for the home directory: `~`
E.g.: `cd ~/Desktop/`
- Returning to the previous working directory:
`cd -`

List files: `ls`

- “ls” (list) it’s a very important command, as it replace the common **file windows**.
- `ls` alone lists the file in my working directory
- As argument I can give **one or more directory**, and “ls” will print their content (e.g.: `ls /tmp`)
- Also files can be arguments, also with **wildcards**
e.g.: `ls *.fa *.fastq`

List files: `ls`

- **Switches** can be combined (`-l -a = -la`)
- `-l` detailed list
- `-a` includes hidden files
- `-h` file size given in *human readable* format
- `man ls` detailed manual
- How can I sort by date? OR by size?

Wildcards

- ? substitutes a single character
- * substitutes any string of characters

- Examples:

```
ls *.fastq
```

```
ls *_R?_*.fastq
```

Characters range

- [a–z] single lowercase letter
- [A–Z] single uppercase letter
- [0–9] single digit
- E.g.: `ls *_R[1-2]*.fastq`

Copy a file `cp`

- Syntax: `cp` `[origin]` `[destination]`
- Origin can be one or more files.
- `-r` to recursively copy directories and their content
- `man cp` as always...

Create directories `mkdir`

- Syntax: `mkdir [newDirName] ...`
- `-p` to create intermediate directories if necessary:
`mkdir -p day1/data/sample`
- `man mkdir` for further details

Remove directories `rmdir`

- Syntax: `rmdir [DirName] ...`
- Only removes empty directories (safe)
- `man rmdir`

Delete files `rm`



- Syntax: `rm [file] ...`
- Removes files, not directories, by default
- `-r` also removes directories and their content (😱)
- Careful with this one, there is no "Trash bin"

How to do it the right way

Before running a dangerous command
eg: `rm ./files/*.gz`

TEST it with an "ls"
e.g.: `ls ./files/*.gz`

View text files `cat`

- Syntax: `cat [file1] [file2]...`
- Will print **all** the content of one or multiple text files
- If the file is very long... can be a problem! (Ctrl-C)

head and tail

- Syntax: `head [file1] [file2]..`
- Will print the first ten lines of a file
- `head -n 16 [file1] [file2]..`
- Similarly, `tail` will print the last lines

man



- Syntax: `man command`
- Manual for shell commands
- You can scroll with **arrow keys** and **quit with q**
- `g` and `G` to quickly reach the top/bottom
- `/` to start searching
- `n` `N` to go to the next / previous occurrence

less

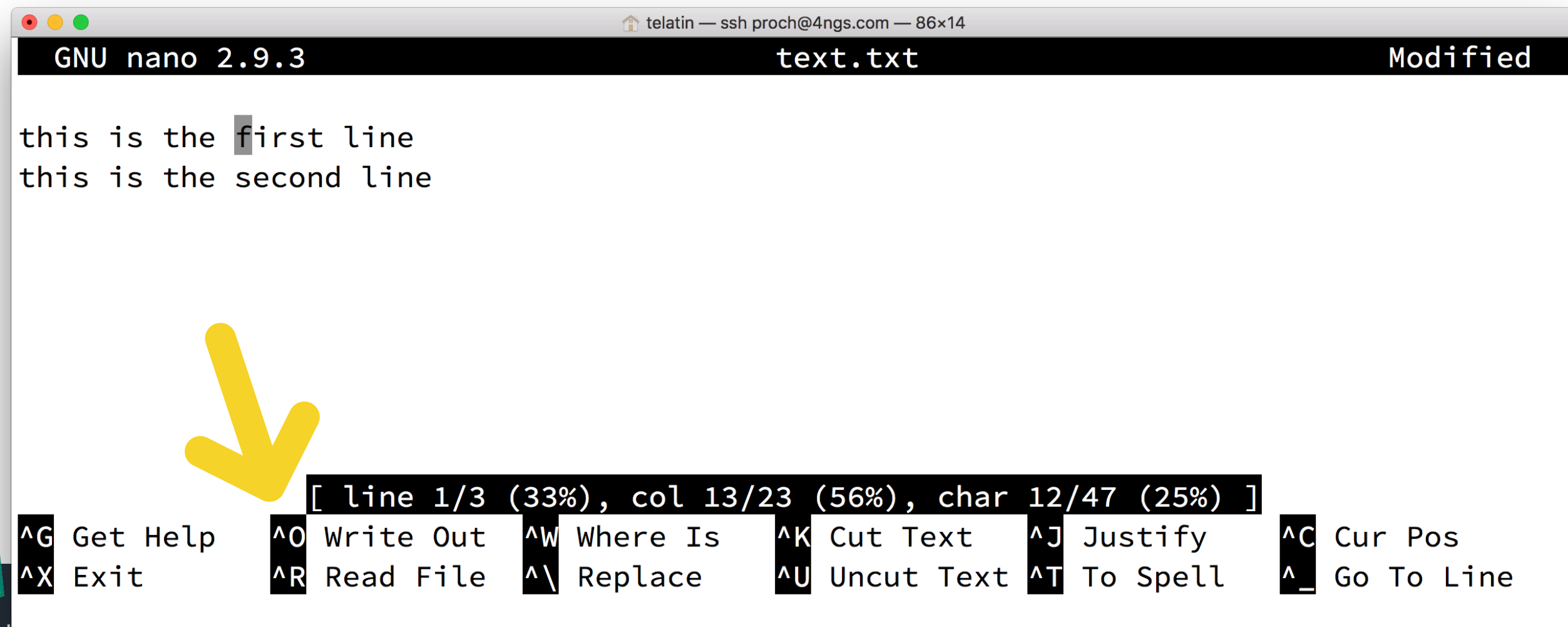


- Syntax: `less [file1]`
- An interactive viewer of text files. You can scroll with **arrow keys** and quit with `q`. Works like *man*!
- To prevent word wrap: `less -S [file1]`
- Change tab size: `less -x 18 [file1]`

nano



- Syntax: `nano [file]`
- An interactive text editor with tool bar [$\wedge$ = Ctrl]



```
telatin — ssh proch@4ngs.com — 86x14
GNU nano 2.9.3 text.txt Modified

this is the first line
this is the second line

[ line 1/3 (33%), col 13/23 (56%), char 12/47 (25%) ]
^G Get Help      ^O Write Out    ^W Where Is    ^K Cut Text    ^J Justify     ^C Cur Pos
^X Exit          ^R Read File    ^\ Replace     ^U Uncut Text  ^T To Spell    ^_ Go To Line
```

wc: word count

lines, words, characters

```
telatin — ~ — ssh qibuser001@learning.seq.space — 78x20
qibuser001@learning: /homes/qib/qibuser001$ wc -l examples/phage/*. *
  2 examples/phage/annotation_hashes.txt
  1 examples/phage/assembly_status.txt
  0 examples/phage/GCF_000840245.1_ViralProj14204
 13 examples/phage/md5checksums.txt
461 examples/phage/README.txt
 25 examples/phage/vir_assembly_report.txt
 68 examples/phage/vir_assembly_stats.txt
670 examples/phage/vir_cds_from_genomic.fna
  5 examples/phage/vir_feature_count.txt
189 examples/phage/vir_feature_table.txt
608 examples/phage/vir_genomic.fna
3213 examples/phage/vir_genomic.gbff
 314 examples/phage/vir_genomic.gff
 290 examples/phage/vir_protein.faa
15698 examples/phage/vir_protein.gpff
1292 examples/phage/vir_rna_from_genomic.fna
 290 examples/phage/vir_translated_cds.faa
23139 total
qibuser001@learning: /homes/qib/qibuser001$
```

Download a file

- Download remote files from web servers (usually works with static URLs only)
- `wget 'url'`
- `curl -o saved.file 'url'`

Other commands

- Find files and directory inside a tree: `find`
- Sorting lines: `sort [-n] [-k N] [file]...`
- Extract lines from a text file using *pattern matching* (more advanced than simple “search”):
`grep "pattern" [FILE]`
- Extract a “column” from a file: `cut -f N -d ' , '`

Further readings

- `grep` regular expressions: [link](#)
- Long presentation with (too many) examples on `grep`, `find`, `awk`: [link](#)
- `Grep`, from a bioinformatician: [link](#)
- Lecture notes on Linux: [link](#)

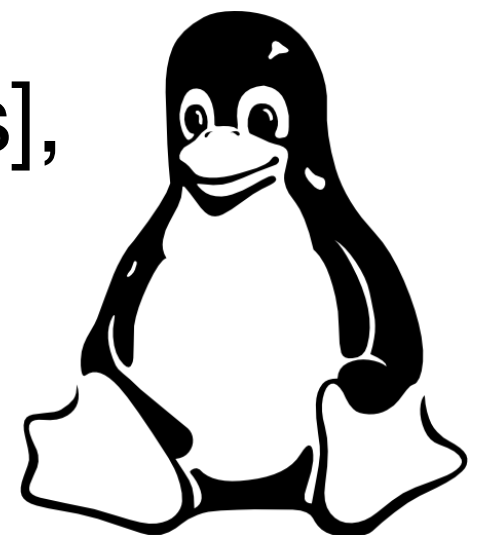


Redirection and pipes

UNIX philosophy

a set of cultural norms and philosophical approaches to minimalist, modular software development.

- Write programs that do one thing and do it well.
- Write programs to work together.
- Write programs to handle text streams [pipes], because that is a universal interface.



Andrea Telatin



Redirection

- Programs often print some text on our terminal. If we want we can redirect that text from the terminal to a file (i.e. save to a file the output)
- `ls -l > my_list.txt`
`find / > all_files.txt 2> errors.log`



Piping commands

A method to create a flow of data: the output of the upstream command becomes the input of the downstream program.

- `find ~ | wc -l`
- `find ~ > list.txt`
`wc -l list.txt`
`rm list.txt`



Piping commands

The first command must work on its own, the subsequent commands should receive a stream of data

```
grep lambda vir_*.gff | grep recomb
```

```
grep lambda vir*.gff | grep recomb vir
```

Miniconda package manager



What is miniconda

- A package manager: makes it easy to install new software
- Allows to create isolated “environments” where to put one or more packages, to prevent conflicts
- Has a bioinformatics “channel” called **bioconda**



Install a package

* <https://docs.conda.io/en/latest/miniconda.html>

- `conda install -c bioconda bedtools samtools`
- `conda search -c bioconda "seqfu>1.10"`

My philosophy

